

PRACTICAL 1

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;
```

// Bubble Sort

```
void bubbleSort(vector<int> &numbers)
{
    int size = numbers.size();

    for (int i = 0; i < size - 1; i++)
    {
        for (int j = 0; j < size - i - 1; j++)
        {
            if (numbers[j] > numbers[j + 1])
            {
                swap(numbers[j], numbers[j + 1]);
            }
        }
    }
}
```

// Selection Sort

```
void selectionSort(vector<int> &numbers)
{
    int size = numbers.size();

    for (int i = 0; i < size - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < size; j++)
        {
            if (numbers[j] < numbers[minIndex])
            {
                minIndex = j;
            }
        }
    }
}
```

PRACTICAL 1

```
    }  
  
    swap(numbers[i], numbers[minIndex]);  
}  
}
```

// Insertion Sort

```
void insertionSort(vector<int> &numbers)  
{  
    int size = numbers.size();  
  
    for (int i = 1; i < size; i++)  
    {  
        int current = numbers[i];  
        int j = i - 1;  
  
        while (j >= 0 && numbers[j] > current)  
        {  
            numbers[j + 1] = numbers[j];  
            j--;  
        }  
  
        numbers[j + 1] = current;  
    }  
}
```

// Merge sort

```
void merge(vector<int> &numbers, int left, int mid, int right)  
{  
    int leftSize = mid - left + 1;  
    int rightSize = right - mid;  
  
    vector<int> leftArray(leftSize), rightArray(rightSize);  
  
    for (int i = 0; i < leftSize; i++)  
    {  
        leftArray[i] = numbers[left + i];  
    }  
  
    for (int j = 0; j < rightSize; j++)
```

PRACTICAL 1

```
{
    rightArray[j] = numbers[mid + 1 + j];
}

int i = 0, j = 0, k = left;

while (i < leftSize && j < rightSize)
{
    if (leftArray[i] <= rightArray[j])
    {
        numbers[k++] = leftArray[i++];
    }
    else
    {
        numbers[k++] = rightArray[j++];
    }
}

while (i < leftSize)
{
    numbers[k++] = leftArray[i++];
}

while (j < rightSize)
{
    numbers[k++] = rightArray[j++];
}
}

void mergeSort(vector<int> &numbers, int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSort(numbers, left, mid);
        mergeSort(numbers, mid + 1, right);

        merge(numbers, left, mid, right);
    }
}
```

PRACTICAL 1

```
}
```

```
// Quick sort
```

```
int partition(vector<int> &numbers, int low, int high)
```

```
{
```

```
    int pivot = numbers[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
    {
```

```
        if (numbers[j] < pivot)
```

```
        {
```

```
            i++;
```

```
            swap(numbers[i], numbers[j]);
```

```
        }
```

```
    }
```

```
    swap(numbers[i + 1], numbers[high]);
```

```
    return i + 1;
```

```
}
```

```
void quickSort(vector<int> &numbers, int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int pivotIndex = partition(numbers, low, high);
```

```
        quickSort(numbers, low, pivotIndex - 1);
```

```
        quickSort(numbers, pivotIndex + 1, high);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    const int size = 100;
```

```
    vector<int> numbers(size), copyNumbers;
```

```
    srand(time(0));
```

```
    for (int i = 0; i < size; i++)
```

```
    {
```

PRACTICAL 1

```
    numbers[i] = rand() % 1000;
}
cout << "Number of Elements = " << size << "\n\n";

auto start = high_resolution_clock::now();

copyNumbers = numbers;
bubbleSort(copyNumbers);

auto stop = high_resolution_clock::now();

cout << "Bubble Sort Time   : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

start = high_resolution_clock::now();

copyNumbers = numbers;
selectionSort(copyNumbers);

stop = high_resolution_clock::now();

cout << "Selection Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

start = high_resolution_clock::now();

copyNumbers = numbers;
insertionSort(copyNumbers);

stop = high_resolution_clock::now();

cout << "Insertion Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

start = high_resolution_clock::now();

copyNumbers = numbers;
```

PRACTICAL 1

```
mergeSort(copyNumbers, 0, size - 1);

stop = high_resolution_clock::now();

cout << "Merge Sort Time    : "
      << duration_cast<microseconds>(stop - start).count()
      << " microseconds\n";

start = high_resolution_clock::now();

copyNumbers = numbers;
quickSort(copyNumbers, 0, size - 1);

stop = high_resolution_clock::now();

cout << "Quick Sort Time    : "
      << duration_cast<microseconds>(stop - start).count()
      << " microseconds\n";

return 0;
}
```